#1)

#1 Dijkstra: Anya to all other points

current: Anya

Neighbors (unvisited): Frog, whale, Penguin, Dog

①

Node	Distance	Previous
Anya	0	
Frog	0.1	Anya
whale	1	Anya
Penguin	2	Anya
Dog	4	Anya

(All other nodes have distance ∞)

② visited: Anya

Priority Queue: All unvisited [AU]

↳ Frog (0.1), whale (1), Pen. (2), Dog (4) ...

③ current: Frog

Neighbors: none

④ visited. Anya, frog

Priority Queue: [AU]

↳ whale (1), Pen. (2), Dog (4) ...

⑤ current: whale

Neighbors: Rhino, elepant, wolf

Node	Distance	Previous
whale	1	Anya
Rhino	2	whale
Elephant	4	whale
wolf	6	whale

⑥ visited: Anya, Frog, whale

Priority Queue: [AU]

↳ Pen (2), Dog (4), Rhino (2), Elephant (4), wolf (6) ...

⑦ current. Penguin

Neighbor Dog

Node	Distance	Previous
Penguin	2	Anya
Dog	4	Anya (unchanged)

⑧ visited: Anya, Frog, whale, Penguin

PQ: [AU]

↳ Dog (4), Rhino (2), Elephant (4), wolf (6) ...

⑧ current: Rhino

Neighbors: Elephant, Panda

Node	Distance	Prev
Rhino	2	whale
Elephant	2.5	Rhino (updated)
Panda	14	Rhino

⑩ visited: Anya, Frog, whale, Penguin, Rhino

PQ: [AU]

↳ Dog (4), Elephant (2.5), wolf (6), Panda (14) ...

⑩ current: Elephant

Neighbor: None

⑪ visited: Anya, Frog, whale, Penguin, Rhino, Elephant

PQ: [AU]

↳ Dog (4), wolf (6), Panda (14) ...

(12) current : Dog
Neighbor: Bunny

Node	Distance	Previous
Dog	4	Anya
Bunny	7	Dog

(13) Visited: Anya, Frog, Whale, Penguin, Rhino, Elephant, Dog
PQ: [AU]
↳ Wolf (6), Panda (14), Bunny (7), ...

(14) current: Wolf
Neighbor: None

(15) visited: Anya, Frog, Whale, Penguin, Rhino, Elephant, Dog, Wolf
PQ: [AU]
↳ Panda (14), Bunny (7), ...

(16) current: Bunny
Neighbor: Bond

Node	Distance	Prev.
Bunny	7	Dog
Bond	12	Bunny

(17) visited: Anya, Frog, Whale, Penguin, Rhino, Elephant, Dog, Wolf, Bunny
PQ: [AU]
↳ Panda (14), Bond (12), ...

(18) current: Bond
Neighbors: Panda

Node	Distance	Prev
Bond	12	Bunny
Panda	13.5	Bond (updated)

(19) visited: Anya, Frog, Whale, Penguin, Rhino, Elephant, Dog, Wolf, Bunny, Bond
PQ: [AU]
↳ Panda (13.5), Sloth (∞)

(20) current: Panda
Neighbors: Sloth

Node	Distance	Prev
Panda	13.5	Bond.
Sloth	14	Panda

(21) visited: Anya, Frog, Whale, Penguin, Rhino, Elephant, Dog, Wolf, Bunny, Bond, Panda
PQ: Sloth (14)

(22) current: Sloth
neighbors: None

(23) visited: Anya, Frog, Whale, Penguin, Rhino, Elephant, Dog, Wolf, Bunny, Bond, Panda, Sloth
PQ: Empty!

Done

#1 (cont)

Final Table

Node	Distance	Previous
✓ Anya	0	
✓ Frog	0.1	Anya
✓ Whale	1	Anya
Penguin	2	Anya
Rhino	2	Whale
Elephant	2.5	Rhino
Dog	4	Anya
Wolf	6	Whale
Bunny	7	Dog
Bond	12	Bunny
Panda	13.5	Bond
Sloth	14	Panda

Paths

Anya

Anya → Frog

Anya → Whale

Anya → Penguin

Anya → Whale → Rhino

Anya → Whale → Rhino → Elephant

Anya → Dog

Anya → Whale → Wolf

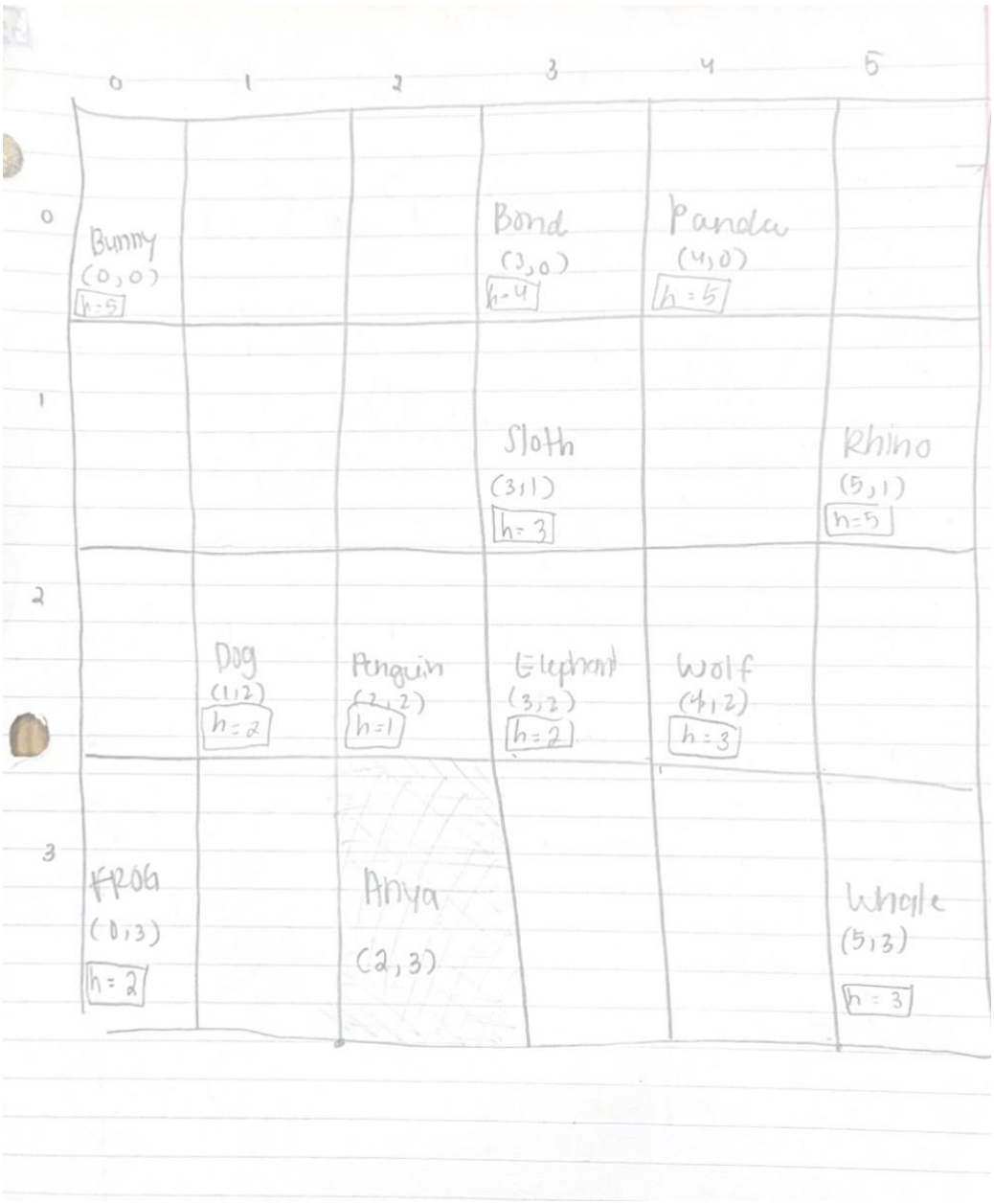
Anya → Dog → Bunny

Anya → Dog → Bunny → Bond

Anya → Dog → Bunny → Bond → Panda

Anya → Dog → Bunny → Bond → Panda → Sloth

#2)



This is what I used to help me find h for the following A* implementation. I know we are going FROM Anya to other points, but because I wanted to keep the h value the same no matter the iteration, I implemented this heuristic as if Anya is the target, and the distance from Anya to the other nodes is the resulting h value (or all nodes are target for Anya, same thing). I wanted to note this, because using this function with a graph doesn't seem like the best option (because it could make more sense to update the h of every neighbor every time). But I just wanted to note that I kept the h values the same for all elements.

Doesn't exactly make sense to use grid, but using Manhattan Distance to find h(n)

#2 | A* See Grid?

① current: Anya

Neighbors: Frog, Whale, Penguin, Dog

Node	g	h	f	prev.
Anya	0	0	0	
Frog	0.1	2	2.1	Anya
Whale	1	3	4	Anya
Penguin	2	1	3	Anya
Dog	4	2	6	Anya

② visited: Anya

PQ: Frog(2.1), Whale(4), Penguin(3), Dog(6)...

③ current: Frog

Neighbors: None

④ visited: Anya, Frog

PQ: Whale(4), Penguin(3), Dog(6)...

⑤ current = Penguin

Neighbors: Dog

↳ Remains the same (g = same → f = same) prev. = Anya

⑥ visited: Anya, Frog, Penguin

PQ: Whale(4), Dog(6)

⑦ current = Whale

Neighbors: Rhino, Elephant, Wolf

Node	g	h	f	prev
Whale	1	3	4	Anya
Rhino	2	5	7	Whale
Elephant	4	2	6	Whale
Wolf	6	3	9	Whale

⑧ visited: Anya, Frog, Penguin, Whale

PQ: Dog(6), Rhino(7), Elephant(6), Wolf(9)

⑨ current = Dog

Neighbors: Bunny

Node	g	h	f	prev
Dog	4	2	6	Anya
Bunny	7	5	12	Dog

⑩ visited: Anya, Frog, Penguin, Whale, Dog

PQ: Rhino(7), Elephant(6), Wolf(9), Bunny(12)

⑪ current: Elephant

Neighbors: None

⑫ visited: Anya, Frog, Penguin, Whale, Dog, Elephant

PQ: Rhino(7), Wolf(9), Bunny(12)

⑬ current: Rhino

Neighbors: Panda

Node	g	h	f	prev
Rhino	2	5	7	Whale
Panda	14	5	19	Rhino

⑭ visited: Anya, Frog, Penguin, Whale, Dog, Elephant, Rhino

PQ: Wolf(9), Bunny(12), Panda(19)

(15) current = Wolf
Neighbors : None

(16) visited: Anya, Frog, Penguin, Whale, Dog, Elephant, Rhino, Wolf
PQ: Bunny(12), Panda(19)

(17) current = Bunny
Neighbors = Bond

Node	g	h	f	prev
Bunny	7	5	12	Dog
Bond	12	5	17	Bunny

(18) visited: Anya, Frog, Penguin, Whale, Dog, Elephant, Rhino, Wolf, Bunny
PQ: Panda(19), Bond(17)

(19) current = Bond
Neighbor : Panda

Node	g	h	f	prev
Bond	12	5	17	Bunny
Panda	13.5	5	18.5	Bond ← update

(20) visited: Anya, Frog, Penguin, Whale, Dog, Elephant, Rhino, Wolf, Bunny, Bond
PQ: Panda(18.5)

(21) current = Panda
Neighbors: Sloth

Node	g	h	f	prev
Panda	13.5	5	18.5	Bond
Sloth	14	3	17	Panda

(22) visited: Anya, Frog, Penguin, Whale, Dog, Elephant, Rhino, Wolf, Bunny, Bond, Panda
PQ: Sloth

(23) current: Sloth
Neighbors : None

(24) visited: All nodes
PQ: None

#2 A*

Final Table

Node	g	h	f	prev.
Anya	0	0	0	
Frog	0.1	2	2.1	Anya
Penguin	2	1	3	Anya
Whale	1	3	4	Anya
Dog	4	2	6	Anya
Elephant	4	2	6	Whale
Rhino	2	5	7	Whale
Wolf	6	3	9	Whale
Bunny	7	5	12	Dog
Bond	12	5	17	Bunny
Panda	13.5	5	18.5	Bond
Sloth	14	3	17	Panda

Anya
Anya → Frog
Anya → Penguin
Anya → Whale
Anya → Dog
Anya → Whale → Elephant
Anya → Whale → Rhino
Anya → Whale → Wolf
Anya → Dog → Bunny
Anya → Dog → Bunny → Bond
Anya → Dog → Bunny → Bond → Panda
Anya → Dog → Bunny → Bond → Panda → Sloth

#3) The order in which the elements are in table correlate to the number of iterations needed to find the shortest path. When using Dijkstra, Anya was able to find a path to the whale (3 vs 4) and rhino (5 vs 7) in less iterations than with A*. When using A*, Anya was able to find a path to the penguin (3 vs 4) and dog with less iterations than with Dijkstra. This is seen with the placement on table, because if it is lower/higher on the table, it took less/more iterations on the table.

#7) Algorithm Analysis

4. The time complexity is O(N^2) where N is the total number of nodes (/vertices) on the graph (/in the matrix). In the program, there is a while loop that iterates until the toCheck queue is empty; this queue will hold all nodes once (so iterates N times). Within the while loop, there is a for loop that iterates through the N times as well. So, the inner loop iterates N times for every iteration of the while loop. The bodies of the loops have constant time complexity, so the loops' time complexity is O(N^2). All other declarations or assignments are constant. Because we can ignore the lower degree time complexities, the final time complexity is O(N^2). The space complexity is O(N^2). This is because the amount of space created is based on the inputted matrix and inputted integer. The visited[] array is of size N, the toCheck queue reaches a max size of (N-1), and within the while loop, int i is declared N times and int j is declared N^2 times. Because the lower degree complexities can be ignored, the space complexity is O(N^2)

5. The time complexity would be O(V*E^3), where Vis the number of vertices on the graph and E is the number of edges on the graph. This is because in cycleFour calls dfsFour() V times, and within dfsFour(), the (recursive case) for loop in the method is recursively called a max of 3 times, because a node that has already been added to the path cannot be visited again (so, at most, dfsFour() is called E(E-1)(E-2)(E-3) times total). So, the time complexity would simplify to O(V*E^3). The space complexity is O(V + E), where V is the total number of vertices and E is the total number of edges on the graph because the number vertices added to the ArrayList in cycleFour is dependent on the number of vertices present in the inputted graph. Additionally, the ArrayList created in dfsFour can contain at most the total number of edges on the graph (if the vertex inputted points to all other vertices in the graph). So, the space complexity is O(V+E).

6. The time complexity is O(N), where N is the total number of vertices in the resulting graph (or in this case the number of rows (or the number of columns, # of rows = # of columns)), because in the program, we are iterating through the created array, which has N elements. Within the for loop, we are doing assignments (with mathematical operations) which would be in constant time. All other operations are of constant time complexity. The space complexity is O(N^2), where N is the total number of vertices in the resulting graph because we are creating arrayS, which has N elements (because it is the total number of space separated strings), and adjMatrix, which has NxN dimensions. Additionally, within the for loop (that iterates N times), int right and left are being created N times. All other operations are constant. This means the resulting space complexity is 3N + N^2. Because we are doing asymptotic analysis we can ignore the lower ordered terms, making O(N^2) our final space complexity.